

ibnIPS Frontend: Step Tracking & Route Building

Phase: Frontend Only (App memory storage, ping backend on final submit)

Target: Add manual step/direction tracking between tagged locations. Allow route preview and editing.

Overview

Extend the Tag Location workflow to capture:

- Steps taken since previous location (manual input)
- Direction of travel (compass buttons: N, NE, E, SE, S, SW, W, NW)
- Route preview with full data
- Edit/delete individual tags
- Submit entire route to backend

Data Flow:

```
Tag Location 1
  ↓ (enter steps, pick direction)
Tag Location 2
  ↓ (enter steps, pick direction)
Tag Location 3
  ↓
[Preview Route]
  ↓
Edit/Delete allowed
  ↓
[Submit to Backend]
```

Storage: App memory only until submit. No persistence between app restarts (for now).

Data Model

Route Collection (In-Memory)

```
route_collection = [  
  {  
    room_id: "Lab_201",  
    room_name: "Physics Lab 201",  
    floor: 2,  
    order: 1,  
    steps_from_previous: null,      // First location has no  
previous  
    direction: null,                // First location has no  
direction  
    timestamp: 1234567890,  
    editable: true  
  },  
  {  
    room_id: "CS_101",  
    room_name: "CS Lab 101",  
    floor: 2,  
    order: 2,  
    steps_from_previous: 42,        // 42 steps from Lab_201 to  
CS_101  
    direction: "NE",                // Traveled northeast  
    timestamp: 1234567920,  
    editable: true  
  },  
  {  
    room_id: "Hall_1F",  
    room_name: "Main Hallway",  
    floor: 2,  
    order: 3,  
    steps_from_previous: 38,  
    direction: "SE",  
    timestamp: 1234567950,  
    editable: true  
  }  
]
```

```
}  
]
```

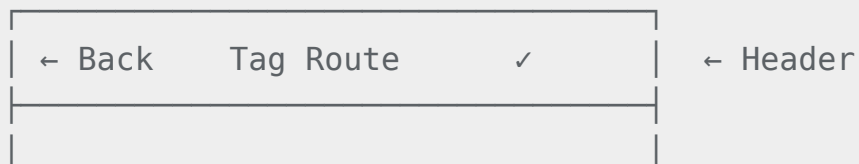
Serialized for Backend Submit

```
{  
  "route": [  
    {  
      "room_id": "Lab_201",  
      "steps_from_previous": null,  
      "direction": null  
    },  
    {  
      "room_id": "CS_101",  
      "steps_from_previous": 42,  
      "direction": "NE"  
    },  
    {  
      "room_id": "Hall_1F",  
      "steps_from_previous": 38,  
      "direction": "SE"  
    }  
  ]  
}
```

Screen: Enhanced Tag Location

Updated Tag Screen (extends existing Tag Location screen)

Layout Structure



Select Room:	
Lab_201 ▼	
Steps from previous:	
[42]	← Text input or stepper
Direction traveled:	
[N] [NW][↑][NE] [W]—————[E] [SW][↓][SE] [S]	← 8-button compass
✓ North (N)	← Selected direction display
Tagged so far: 3 locations	
[Continue Tagging] [Preview]	

Components

Room Selector (Existing, No Change)

- Dropdown showing available rooms
- Defaults to empty/hint
- Required field

Steps Input

- Label: "Steps from previous:"

- Single text input field (number only)
- Placeholder: "e.g., 42"
- Validation: positive integer, max 9999
- Note: Only enabled if NOT first room in route
- Disabled state for first tag: "—" (dash) or grayed out

Direction Buttons (New)

- Label: "Direction traveled:"
- 8 buttons arranged in compass pattern:

```

      [ N ]   [ NW ] [ NE ]   [ W ]   [ E ]   [ SW ] [ SE ]
    [ S ]
  
```

- Button size: 40–48px each
- Visual indication of selected button (color change, outline, or filled state)
- Only enabled if NOT first room in route
- Disabled state for first tag: all buttons grayed out

Direction Display (New)

- Shows selected direction as text + icon
- Format: "✓ Northeast (NE)" or "↗ Northeast"
- Updates as buttons are tapped
- Clears when a different button is pressed

Route Summary

- Label: "Tagged so far: X locations"
- Simple counter
- Updates in real-time as locations are added

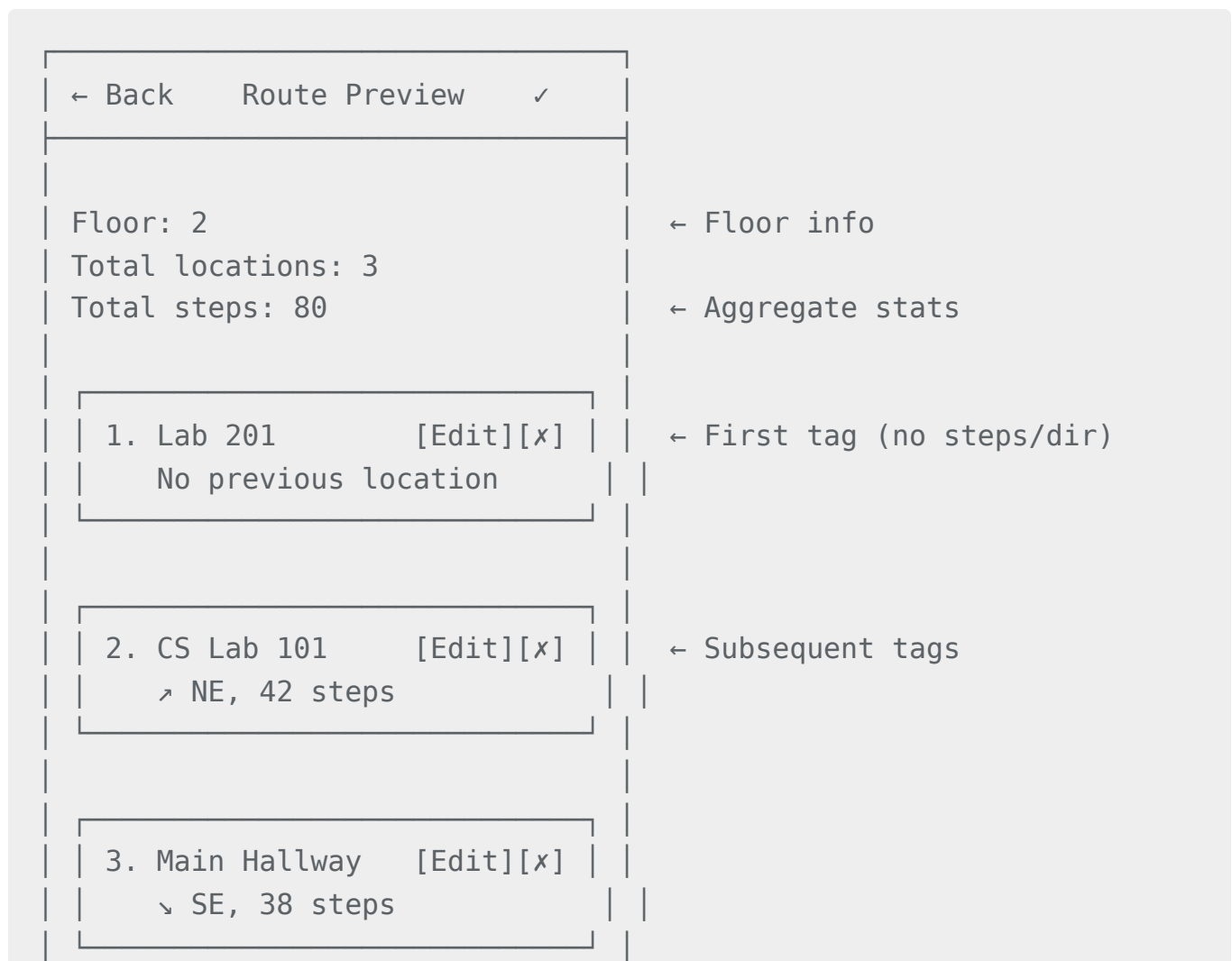
Action Buttons

- Button 1: "Continue Tagging" (blue, primary)
 - Adds current room + steps + direction to route_collection
 - Clears form for next input
 - Disables step/direction fields and grays out buttons (ready for next tag)
- Button 2: "Preview Route" (gray, secondary)
 - Opens RoutePreviewScreen (see below)
 - Does NOT save current input yet

Screen: Route Preview

Purpose: Show all tagged locations, edit/delete individual entries, submit to backend

Layout Structure



[Cancel] [Submit Route]

Components

Header Stats

- Floor level (all tags must be same floor — enforced elsewhere)
- Total locations tagged
- Total steps summed across all hops

Route Entry Cards

- One card per tagged location, in order (1, 2, 3, ...)
- Card layout:
 - **Order number + room name** (bold, 14px)
 - **Edit button** (pencil icon or text "Edit")
 - **Delete button** (× or trash icon)
 - **For first location only:** "No previous location" (grayed text)
 - **For subsequent locations:** "↗ [Direction], [Steps] steps" (normal text, 12px)

Action Buttons

- Button 1: "Cancel" (secondary, red outline or text)
 - Clears entire route_collection
 - Returns to Map Screen
- Button 2: "Submit Route" (primary, blue)
 - Serializes route_collection to JSON
 - Sends HTTP POST to backend (ping only for now)
 - Shows loading spinner during send

- On success: confirmation toast, clears route, returns to Map
 - On error: error toast, stays on preview
-

Interactions: Edit Route Entry

When user taps [Edit] on a card:

1. **Modal or Full Screen:** Opens edit form for that specific entry
 2. **Editable Fields:**
 - Room selector (change room)
 - Steps input (change step count)
 - Direction buttons (change direction)
 3. **On Save:**
 - Update that entry in route_collection
 - Re-validate: if steps changed, no recalc needed (just update)
 - If room changed, update room_name and room_id
 - Return to preview screen
 4. **On Cancel:**
 - Discard changes, return to preview
-

Interactions: Delete Route Entry

When user taps [×/Delete] on a card:

1. **Confirmation Dialog:**

```
"Remove [Room Name] from route?"  
[Cancel] [Remove]
```

2. **On Confirm:**
 - Remove entry from route_collection
 - **Auto-recalculate:**

- The next entry in the chain now becomes the "second location"
- If deleted entry was the second location, the third location (if exists) inherits its step count?
- **OR** the next location's step count becomes null (reset)?
- **Recommendation:** Remove next location's step/direction (reset to null) — requires user to re-enter when they re-tag
- Rebuild route_collection with new order
- Return to preview with updated list

3. On Cancel:

- Keep entry, stay on preview

Interactions: Submit Route

POST /api/ping

```
{
  "route": [
    {
      "room_id": "Lab_201",
      "steps_from_previous": null,
      "direction": null
    },
    {
      "room_id": "CS_101",
      "steps_from_previous": 42,
      "direction": "NE"
    },
    {
      "room_id": "Hall_1F",
      "steps_from_previous": 38,
      "direction": "SE"
    }
  ]
}
```

Response (Expected):

```
{
  "status": "pong",
  "route_received": 3,
  "message": "Route data received (not yet stored)"
}
```

On Success:

- Toast: "Route submitted successfully"
- Clear route_collection (app memory)
- Reset screens
- Return to Map Screen

On Failure:

- Toast: "Failed to submit route. Retry?"
- Keep route_collection in memory
- Offer retry button

State Management

Route Collection State

Variable: routeCollection: RouteEntry[]

Operations:

```
addToRoute(room_id, room_name, floor):
  - Create new entry
  - Set order = routeCollection.length + 1
```

- If first entry: `steps_from_previous = null`, `direction = null`
- If subsequent: `steps_from_previous` and `direction = PENDING` (not saved yet)
- Push to `routeCollection`

`updateRoute(index, { steps, direction }):`

- Find entry at index
- Update `steps_from_previous` and `direction`
- Validate

`deleteFromRoute(index):`

- Remove entry at index
- If removed entry had dependent (next entry):
 - Reset next entry's `steps` and `direction` to null (or remove)
- Re-order all entries (1, 2, 3, ...)
- Return updated `routeCollection`

`getRoute():`

- Return current `routeCollection`

`clearRoute():`

- Reset `routeCollection = []`

Validation Rules

For each entry:

- `room_id` must be non-empty string
- `steps_from_previous` must be positive integer or null (for first entry)
- `direction` must be one of: ["N", "NE", "E", "SE", "S", "SW", "W", "NW"] or null (for first entry)
- All entries on route must be same floor (enforced at preview time)

For route as a whole:

- Minimum 2 locations required to submit (single location is not a route)
- All entries must pass individual validation

UI Components to Build

Existing (No Change)

- ☐ Room selector dropdown
- ☐ Text input (adapt for steps)

New Components

- ☐ **DirectionButtonGroup** — 8-button compass layout
 - Props: selectedDirection, onSelect(direction), disabled
 - Renders buttons in compass pattern
 - Visual feedback on selected button
- ☐ **DirectionDisplay** — Shows selected direction as text + icon
 - Props: direction ("N", "NE", etc.)
 - Renders: "✓ North (N)" or similar
- ☐ **RouteEntryCard** — Individual route location card
 - Props: entry, onEdit, onDelete
 - Renders: order + room name + steps + direction + buttons
- ☐ **RoutePreviewList** — Container for all route cards
 - Props: routeCollection, onEdit, onDelete
 - Renders: header stats + cards
- ☐ **ConfirmDialog** — Reusable modal for confirmation
 - Props: title, message, onConfirm, onCancel
 - Renders: modal with two buttons

Navigation Changes

Screen Stack:

```
MapScreen (home)
├─ TagLocationScreen (modal)
│   └─ RoutePreviewScreen (full screen, navigates back to
MapScreen)
├─ DebugScreen
└─ WifiDebugScreen
```

Flow:

1. From MapScreen, tap "Tag Location" → open TagLocationScreen
 2. From TagLocationScreen, tap "Preview Route" → push RoutePreviewScreen
 3. From RoutePreviewScreen, tap "Submit Route" → POST to backend → pop back to MapScreen
 4. From RoutePreviewScreen, tap "Cancel" → pop back to MapScreen (discard route)
-

Acceptance Criteria

Tag Screen

- ☐ Room dropdown works (no change)
- ☐ Steps input only shows for non-first locations (disabled for first)
- ☐ Direction buttons appear in compass pattern (N, NE, E, SE, S, SW, W, NW)
- ☐ Only one direction button can be selected at a time
- ☐ Direction display updates as buttons are tapped
- ☐ "Continue Tagging" adds location to route_collection and clears form
- ☐ "Preview Route" opens RoutePreviewScreen without saving current input
- ☐ Route counter updates in real-time

Preview Screen

- ☐ All tagged locations appear in order

- ☐ First location shows "No previous location" (no steps/direction)
- ☐ Subsequent locations show "↗ NE, 42 steps" format
- ☐ Floor and total step count displayed correctly
- ☐ [Edit] button opens edit modal for that entry
- ☐ [Delete] button shows confirmation and removes entry
- ☐ Auto-recalc: when entry deleted, next entry's steps/direction reset
- ☐ "Submit Route" serializes and POSTs to backend
- ☐ Success/error toasts appear on submit
- ☐ "Cancel" discards route and returns to Map

Data Integrity

- ☐ routeCollection persists during user navigation (in memory)
- ☐ routeCollection clears after successful submit
- ☐ Minimum 2 locations required to submit
- ☐ All entries must be same floor
- ☐ Step counts are positive integers (validated)
- ☐ Directions are from 8-direction set (validated)

Error Handling

- ☐ Backend unreachable: show retry toast, keep route in memory
 - ☐ Invalid input: show validation error, don't add to route
 - ☐ Network timeout (5s): show "Unable to submit. Retry?"
-

Deliverables Timeline

Week 1:

- ☐ DirectionButtonGroup component built and styled
- ☐ DirectionDisplay component working

- ☐ routeCollection state management implemented

Week 2:

- ☐ TagLocationScreen extended (steps input, direction picker)
- ☐ "Continue Tagging" button adding to route
- ☐ Route counter displaying correctly

Week 3:

- ☐ RoutePreviewScreen full implementation
- ☐ RouteEntryCard component for individual entries
- ☐ Edit and delete interactions working

Week 4:

- ☐ Backend POST /api/ping integrated
- ☐ Error handling and toasts
- ☐ Full flow tested (tag → preview → submit → clear)
- ☐ UI polish and refinements

File Structure

```
screens/  
├─ TagLocationScreen.tsx      (UPDATED)  
└─ RoutePreviewScreen.tsx    (NEW)  
  
components/  
├─ DirectionButtonGroup.tsx  (NEW)  
├─ DirectionDisplay.tsx      (NEW)  
├─ RouteEntryCard.tsx        (NEW)  
├─ RoutePreviewList.tsx      (NEW)  
└─ ConfirmDialog.tsx         (NEW, if not exists)
```

```
└─ ...existing

services/
├─ routeService.ts           (NEW)
├─ apiClient.ts              (UPDATED: add /api/ping)
└─ wifiService.ts            (no change)

hooks/
├─ useRoute.ts                (NEW: state management)
└─ ...existing

types/
└─ route.ts                   (NEW: RouteEntry type defs)
```

Validation Examples

Valid Route Submit:

```
{
  "route": [
    { "room_id": "Lab_201", "steps_from_previous": null,
      "direction": null },
    { "room_id": "CS_101", "steps_from_previous": 42, "direction":
      "NE" },
    { "room_id": "Hall_2F", "steps_from_previous": 35, "direction":
      "S" }
  ]
}
```

Invalid Route (blocked before submit):

- Only 1 location → Error: "Route requires at least 2 locations"
- Steps = -5 → Error: "Steps must be positive"
- Direction = "N" but first location → Error: "First location cannot have direction"

- Mixed floors (some on floor 1, some on floor 2) → Error: "All locations must be on same floor"
-

Notes

- **No backend storage yet.** Route data sent to `/api/ping` is just a ping — backend doesn't persist it.
 - **App memory only.** Closing app clears `route_collection`.
 - **Manual only.** No pedometer, no accelerometer, no auto-count. User enters steps manually.
 - **Editable before submit.** After submit, route is cleared. No history kept in app.
 - **Direction is text.** 8 cardinal/intercardinal directions only. User taps button, direction stored as "N", "NE", etc.
-

This is the frontend roadmap for route building. Backend will handle aggregation, verification, and storage after launch.